

REST API design

COMPLETE LEARNING GUIDE

What You'll Learn

REST (Representational State Transfer) is a set of architectural constraints — not a protocol — that, applied well, makes an API predictable enough that a developer who has never seen your specific endpoints can guess how they behave. This guide covers designing resource-oriented URIs, using HTTP methods according to their actual semantics (especially idempotency, which many production bugs violate), choosing status codes that communicate real outcomes instead of defaulting to 200 for everything, structuring request/response bodies consistently, and the practical concerns that separate a toy API from a production one: pagination for large collections, a versioning strategy for breaking changes, and consistent, non-leaky error responses. These conventions matter because they're what every frontend, mobile client, and third-party integrator relies on implicitly.

1. Resources and URI Design

REST models everything as a resource — a noun, not a verb — identified by a URI. Collections are plural nouns (`/students`), and a specific item within a collection is addressed by appending its identifier (`/students/42`). Nesting expresses ownership or containment (`/students/42/enrollments` for that student's enrollments), but nesting should stay shallow — more than two or three levels deep becomes unwieldy and usually signals the nested resource deserves its own top-level, filterable endpoint instead (`/enrollments?studentId=42`).

- URIs should never contain verbs like `/getStudents` or `/createStudent` — the HTTP method already conveys the action.
- Use plural nouns consistently for collections (`/students`, not `/student`) to avoid inconsistent pluralization across the API.
- Query parameters are for filtering, sorting, and pagination (`?status=active&sort=-createdAt`), not for identifying a specific resource.

2. HTTP Methods and Idempotency

Each HTTP method carries a specific meaning that clients, proxies, and caches all rely on. GET retrieves and must never cause a side effect. POST creates a new resource (or triggers a non-idempotent action) and is not idempotent — calling it twice can create two resources. PUT replaces a resource entirely and is idempotent — calling it *n* times has the same effect as calling it once. PATCH applies a partial update and may or may not be idempotent depending on implementation. DELETE removes a resource and should be idempotent (deleting an already-deleted resource should not error in a way that breaks retries). Violating idempotency — e.g., a PUT that increments a counter rather than setting an absolute value — breaks client

retry logic in subtle, hard-to-debug ways.

3. Status Codes That Communicate Intent

Returning 200 OK for every response, with success/failure buried in the response body, forces every client to parse the body just to know if a request worked — defeating a huge part of what HTTP already gives you for free. 2xx codes indicate success (200 OK, 201 Created for a successful POST, 204 No Content when there's nothing to return, e.g. after a DELETE). 4xx indicates the client did something wrong (400 Bad Request for malformed input, 401 Unauthorized for missing/invalid auth, 403 Forbidden for valid auth but insufficient permission, 404 Not Found, 409 Conflict for a state clash like a duplicate unique field). 5xx indicates the server failed (500 Internal Server Error, 503 Service Unavailable). Using these precisely lets clients branch on status code alone before ever inspecting the body.

4. Request/Response Body Design

JSON is the default body format for modern REST APIs, and consistency across endpoints matters more than any specific convention chosen — pick one casing style (commonly camelCase for JSON) and use it everywhere, pick one date format (ISO 8601, e.g. 2026-09-18T14:30:00Z) and use it everywhere, and wrap list responses in a consistent envelope (e.g. { data: [...], meta: { total, page } }) rather than sometimes returning a bare array and sometimes an object. The Content-Type header (application/json) and, on the request side, honoring Accept headers for content negotiation, round out a well-behaved API.

JAVASCRIPT

```
app.get("/api/students", async (req, res) => {
  const page = Number(req.query.page) || 1;
  const limit = Math.min(Number(req.query.limit) || 20, 100);

  const [students, total] = await Promise.all([
    db.student.findMany({ skip: (page - 1) * limit, take: limit }),
    db.student.count(),
  ]);

  res.status(200).json({
    data: students,
    meta: { page, limit, total, totalPages: Math.ceil(total / limit) },
  });
});
```

5. Pagination, Filtering, and Sorting

Any endpoint that can return an unbounded number of records must be paginated — returning all 500,000 rows because nobody added a limit is a classic production incident waiting to happen. Offset-based pagination (?page=2&limit=20) is simple and sufficient for most admin/UI use cases; cursor-based pagination (?after=<opaque_cursor>) scales better for very large or frequently-changing datasets because it avoids the 'skipped or duplicated row' problem that occurs when rows are inserted/deleted between paginated requests using offsets. Filtering (?status=active) and sorting (?sort=-createdAt, with a - prefix for descending) should follow one consistent query-parameter convention across every

collection endpoint in the API.

6. API Versioning Strategies

Once an API has real consumers, you can't silently make a breaking change (removing a field, changing a type, renaming a route) without breaking them. The two dominant strategies are URI versioning (`/api/v1/students`, `/api/v2/students`) — simple, highly visible, but means maintaining parallel route trees — and header-based versioning (a custom `Accept` or `API-Version` header), which keeps URIs stable but is less discoverable and harder to test casually in a browser. Whichever is chosen, the real discipline is defining what counts as 'breaking' (removing/renaming a field, changing a type, changing status code semantics) versus safe to add without a version bump (adding a new optional field).

7. Error Response Design

A consistent error shape lets every client handle failures the same way instead of writing bespoke parsing per endpoint. A good baseline includes a machine-readable error code, a human-readable message, and — for validation errors — a list of which specific fields failed and why. Stack traces and internal implementation details should never reach the client in production; they belong in server-side logs, not the response body, both because they leak information useful to an attacker and because they're meaningless to the API consumer.

JSON

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request failed validation",
    "details": [
      { "field": "email", "issue": "must be a valid email address" }
    ]
  }
}
```

Common Pitfalls

- Putting verbs in URIs (`/getStudent`, `/createStudent`) instead of letting the HTTP method carry the action.
- Returning 200 OK for failed requests with the actual error buried in the JSON body, forcing every client to parse the body to detect failure.
- Implementing PUT in a way that isn't idempotent (e.g. incrementing rather than replacing), which silently breaks safe client retries.
- Shipping a collection endpoint with no pagination at all, which works fine in testing and becomes an incident once the table has real production volume.
- Inconsistent casing or date formats across different endpoints in the same API, forcing clients to special-case each one.

- Making a breaking change to a live endpoint with no versioning strategy, breaking every existing consumer with no warning.
- Leaking stack traces or internal error details in API responses instead of logging them server-side and returning a generic client-safe message.

Key Takeaways

- URIs name resources (nouns); HTTP methods express the action — mixing the two produces an API that's harder to predict.
- GET/PUT/DELETE should be idempotent; POST is not — client retry logic depends on this distinction being respected.
- Precise status codes let clients branch on the response before ever parsing the body.
- Any collection endpoint that can grow unbounded must be paginated from day one, not retrofitted after an incident.
- Decide your versioning and breaking-change policy before you have real API consumers, not after the first breaking change ships.
- Error responses should be consistent and client-safe; stack traces belong in logs, not in the HTTP response.

Learn More

MDN — HTTP Request Methods

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>

MDN — HTTP Response Status Codes

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>

Microsoft REST API Guidelines

<https://github.com/microsoft/api-guidelines/blob/vNext/azure/Guidelines.md>